Practical in Python for Business Intelligence

1. Data Collection from a CSV File:
- Use Pandas to read and load data from a CSV file.
- Perform initial data exploration and visualization using Matplotlib or Seaborn.

Solution:

```
# Import the necessary libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

 # Step 1: Read and Load Data from a CSV File
 # Replace 'your_data.csv' with the path to your CSV file
data = pd.read_csv(r'C:\Users\khana\OneDrive\Desktop\Sana\IT\TY IT\BI\NEw
Practical\Sample.csv')

 # Step 2: Initial Data Exploration
 # Display the first few rows of the dataset
print("First 5 rows of the dataset:")
print(data.head())

 # Get basic statistics of the numerical columns
print("\nSummary statistics of numerical columns:")
print(data.describe())

 # Get information about the dataset, including data types and missing values
print("\nData information:")
print(data.info())

 # Data Visualization
 # Example data visualization: Create a histogram of a numerical column
plt.figure(figsize=(8, 6))
sns.histplot(data['Age'], bins=20, kde=True)
plt.title('Histogram of Age')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()

 # Example data visualization: Create a boxplot of a numerical column
plt.figure(figsize=(8, 6))
sns.boxplot(data=data, x='Gender', y='Age')
plt.title('Boxplot of Age by Gender')
plt.xlabel('Gender')
plt.ylabel('Age')
plt.xticks(rotation=45)
plt.show()
```

Alice	30	Female	50000	Master's
Bob	25	Male	60000	Bachelor's
Charlie	35	Male	75000	Ph.D.
David	28	Male	45000	Bachelor's
Eve	32	Female	55000	Master's
Frank	22	Male	42000	Bachelor's
Grace	29	Female	70000	Ph.D.
Hannah	27	Female	52000	Bachelor's
Isaac	40	Male	80000	Master's

OUTPUT:

```
Summary statistics of numerical columns:
             Age        Salary
count   9.000000      9.000000
mean   29.777778  58777.777778
std     5.380004  13460.848578
min    22.000000  42000.000000
25%    27.000000  50000.000000
50%    29.000000  55000.000000
75%    32.000000  70000.000000
max    40.000000  80000.000000

Data information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Name       9 non-null      object
 1   Age        9 non-null      int64
 2   Gender     9 non-null      object
 3   Salary     9 non-null      int64
 4   Education  9 non-null      object
dtypes: int64(2), object(3)
memory usage: 492.0+ bytes
None
```

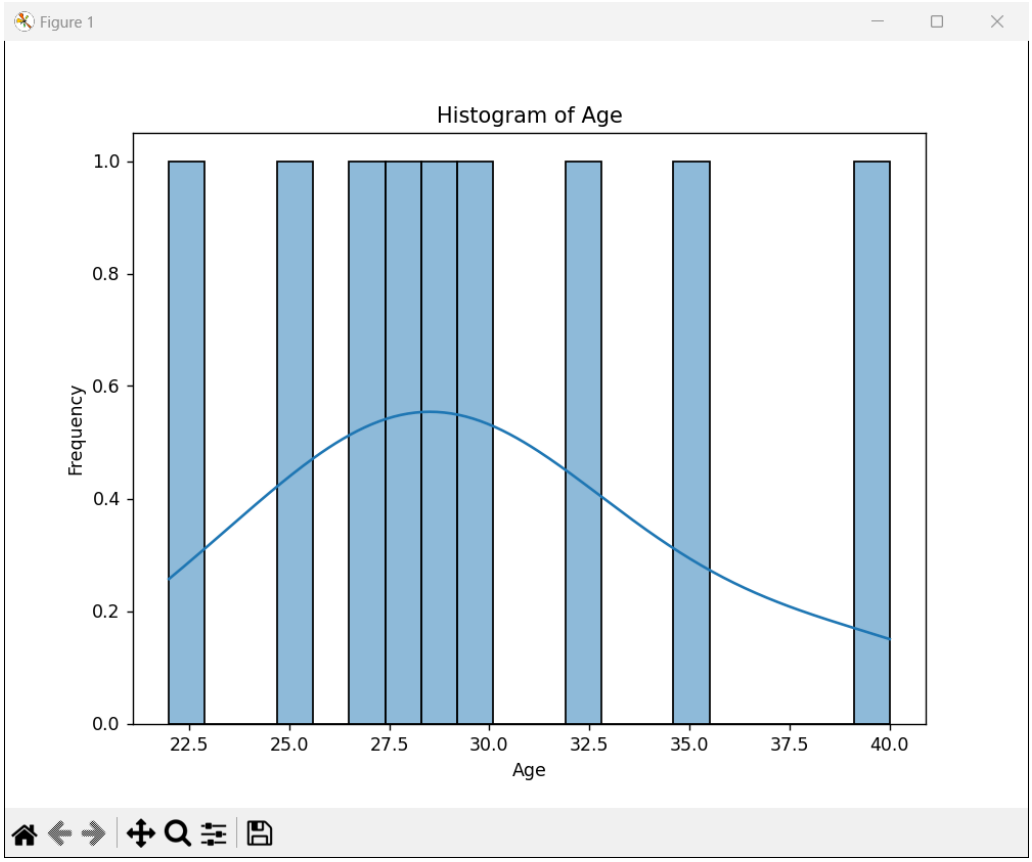


Figure 1
Histogram of Age
Frequency
1.0
0.8
0.6
0.4
0.2
0.0
22.5
25.0
27.5
30.0
32.5
35.0
37.5
40.0
Age

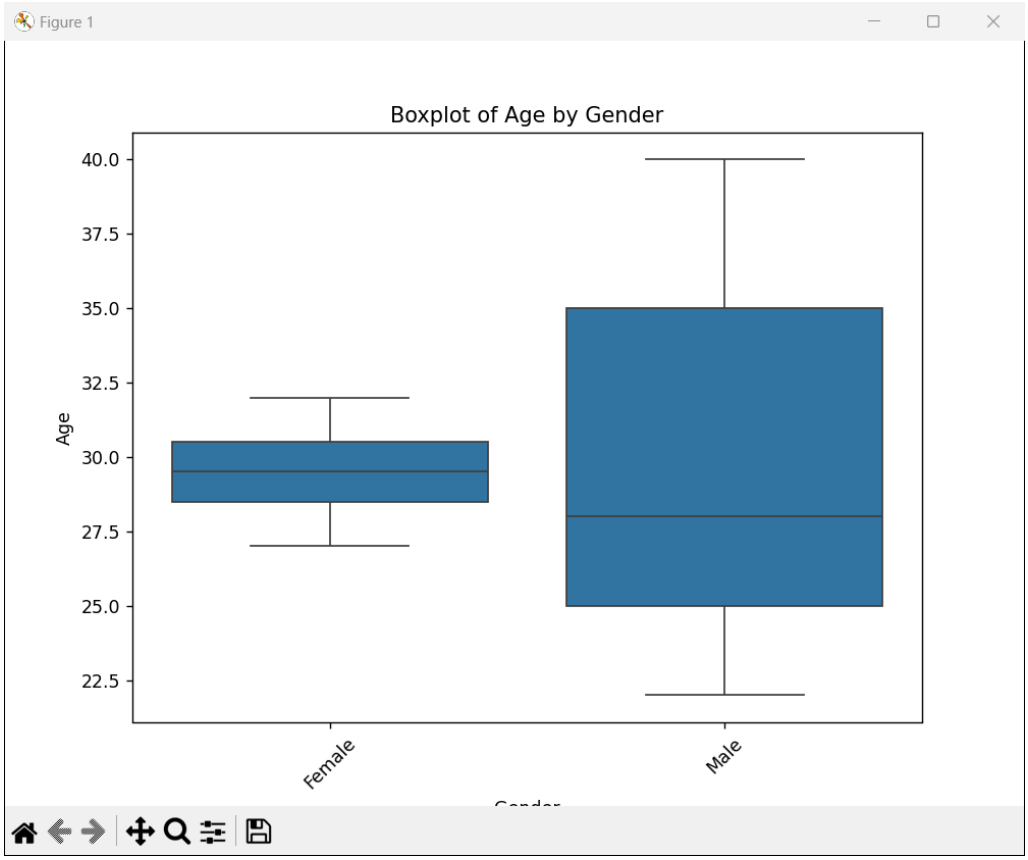


Figure 1
Boxplot of Age by Gender
Age
40.0
37.5
35.0
32.5
30.0
27.5
25.0
22.5
Female
Male

Or

(Loading TITANIC data set)

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the Titanic dataset
titanic_url =
"https://web.stanford.edu/class/archive/cs/cs109/cs109.1166/stuff/titanic.csv"
titanic = pd.read_csv(titanic_url)

# Display the first few rows of the dataset
print("First 5 rows of the Titanic dataset:")
print(titanic.head())

# Basic Information about the dataset
print("\nBasic Information about the Titanic dataset:")
print(titanic.info())

# Summary statistics of numerical columns
print("\nSummary statistics of the Titanic dataset:")
print(titanic.describe())

# Visualize the distribution of passenger ages using a histogram
plt.figure(figsize=(8, 6))
sns.histplot(titanic['Age'].dropna(), bins=30, kde=True)
plt.title('Distribution of Passenger Ages')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()

# Visualize the count of passengers in each class using a bar plot
plt.figure(figsize=(8, 6))
sns.countplot(x='Pclass', data=titanic, palette='viridis')
plt.title('Count of Passengers in Each Class')
plt.xlabel('Class')
plt.ylabel('Count')
plt.show()

# Visualize the survival status of passengers using a bar plot
plt.figure(figsize=(8, 6))
sns.countplot(x='Survived', data=titanic, palette='muted')
plt.title('Survival Status of Passengers')
plt.xlabel('Survived (0 = No, 1 = Yes)')
plt.ylabel('Count')
plt.show()
```

OUTPUT:

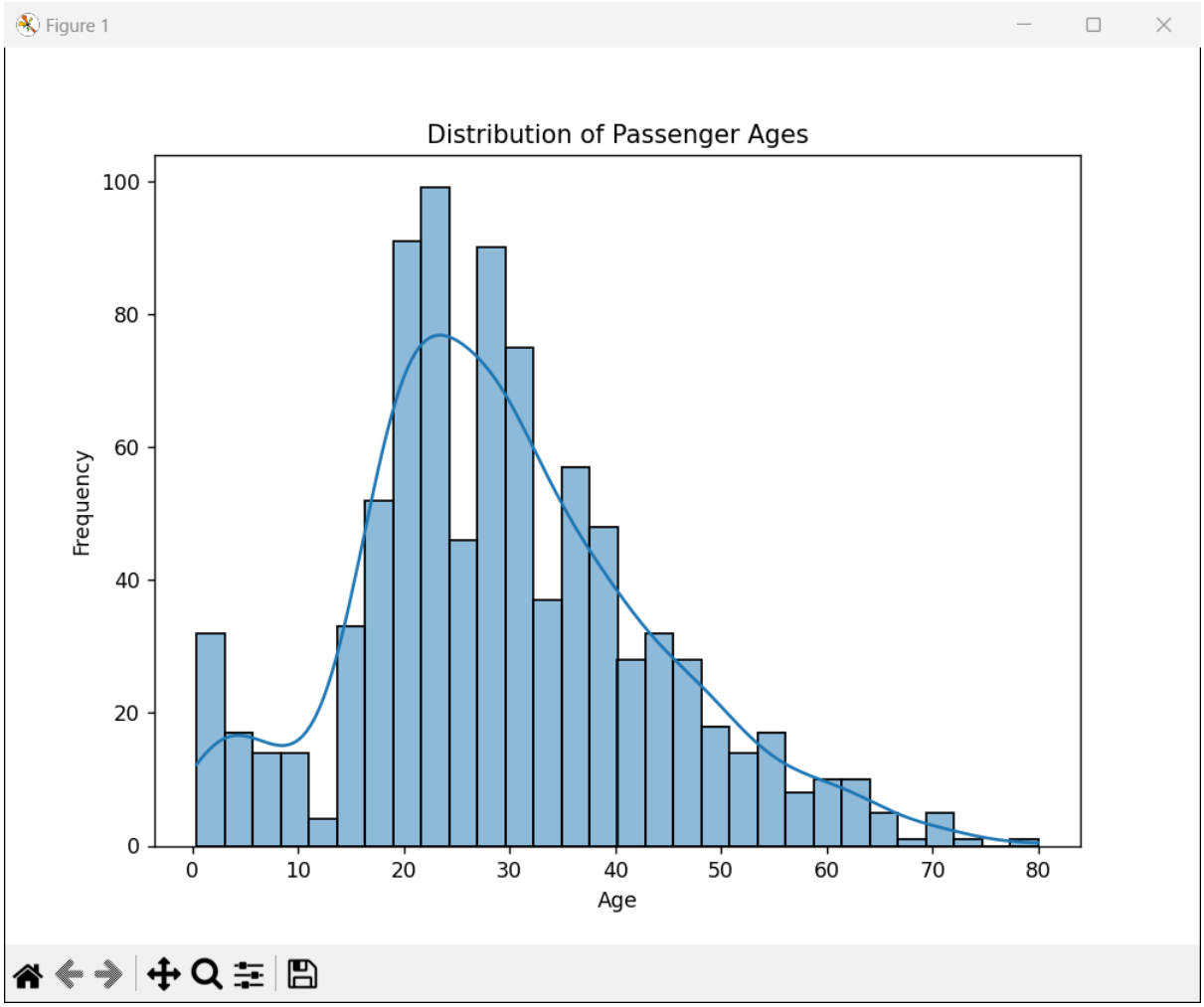
```
PS C:\Users\khana> & C:/Users/khana/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/khana/OneDrive/Desktop/Sana/IT/TY IT/BI/NEw Practical/P1_2.py"
First 5 rows of the Titanic dataset:
   Survived  Pclass  ... Parents/Children Aboard     Fare
0         0       3  ...                      0   7.2500
1         1       1  ...                      0  71.2833
2         1       3  ...                      0   7.9250
3         1       1  ...                      0  53.1000
4         0       3  ...                      0   8.0500

[5 rows x 8 columns]

Basic Information about the Titanic dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 887 entries, 0 to 886
Data columns (total 8 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Survived                 887 non-null    int64
 1   Pclass                   887 non-null    int64
 2   Name                     887 non-null    object
 3   Sex                      887 non-null    object
 4   Age                      887 non-null    float64
 5   Siblings/Spouses Aboard  887 non-null    int64
 6   Parents/Children Aboard  887 non-null    int64
 7   Fare                     887 non-null    float64
dtypes: float64(2), int64(4), object(2)
memory usage: 55.6+ KB
None

Summary statistics of the Titanic dataset:
          Survived      Pclass  ...  Parents/Children Aboard       Fare
count  887.000000  887.000000  ...               887.000000  887.00000
mean     0.385569    2.305524  ...                 0.383315   32.30542
std      0.487004    0.836662  ...                 0.807466   49.78204
min      0.000000    1.000000  ...                 0.000000    0.00000
25%      0.000000    2.000000  ...                 0.000000    7.92500
50%      0.000000    3.000000  ...                 0.000000   14.45420
75%      1.000000    3.000000  ...                 0.000000   31.13750
max      1.000000    3.000000  ...                 6.000000  512.32920

[8 rows x 6 columns]
```



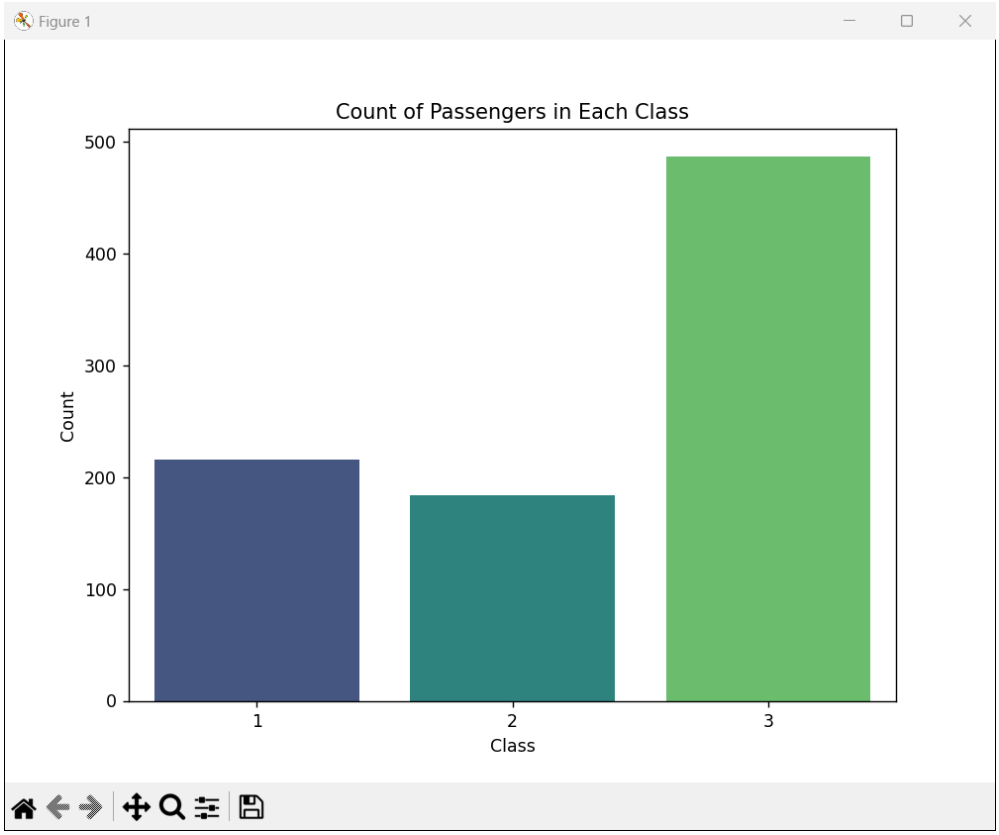


Figure 1
Count of Passengers in Each Class
500
400
300
200
100
0
Count
1
2
3
Class

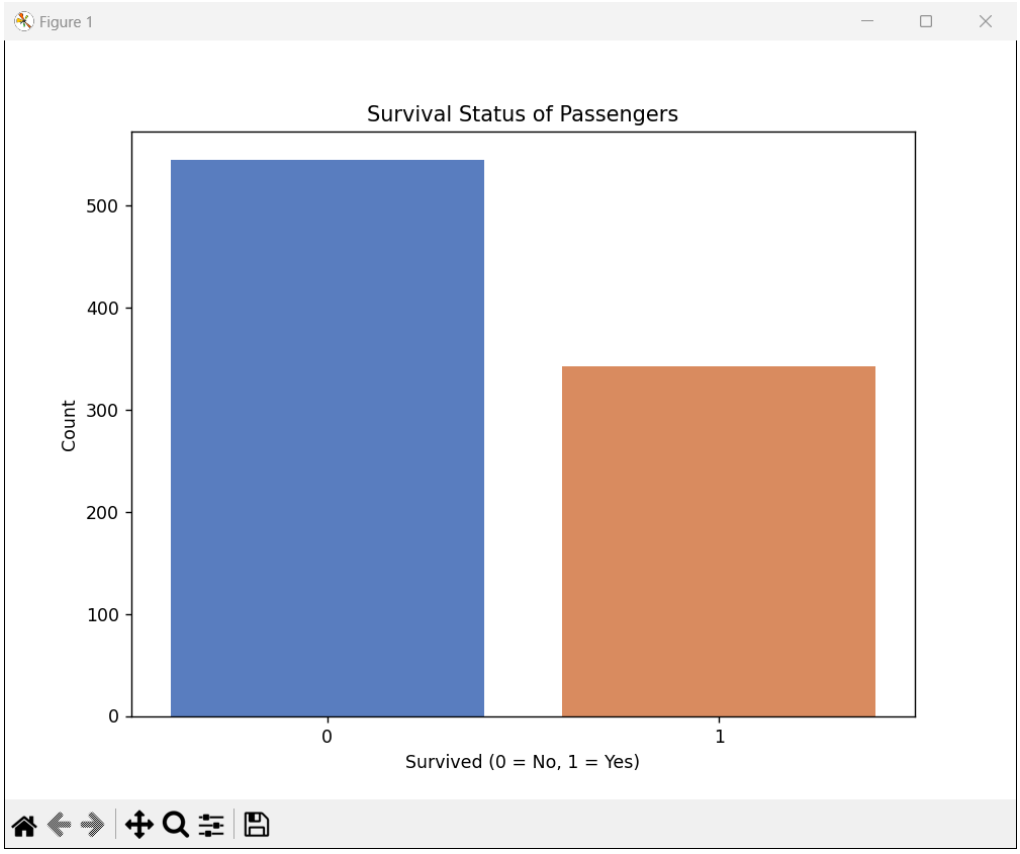


Figure 1
Survival Status of Passengers
500
400
300
200
100
0
Count
0
1
Survived (0 = No, 1 = Yes)

2 Custom Data Collection via Web Scraping:

- Use Python libraries like BeautifulSoup and Requests to scrape data from websites or APIs.

- Preprocess and analyze the scraped data for business insights.

Code:

```
# Import the necessary libraries
import requests
from bs4 import BeautifulSoup
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Read and Load Data from the Web
# Define the URL of the website to scrape
url = 'http://quotes.toscrape.com/'

# Send an HTTP GET request to the URL
response = requests.get(url)

# Parse the HTML content of the page
soup = BeautifulSoup(response.text, 'html.parser')


# Extract all the quotes and their authors
quotes = []
authors = []
for quote in soup.find_all('span', class_='text'):
 quotes.append(quote.get_text())
for author in soup.find_all('small', class_='author'):
 authors.append(author.get_text())

# Create a DataFrame from the extracted data
data_df = pd.DataFrame({'Quote': quotes, 'Author': authors})

# Example data visualization: Create a bar chart to show the number of quotes
by author
author_counts = data_df['Author'].value_counts()
# Example data visualization: Create a bar chart to show the number of quotes
by author
plt.figure(figsize=(10, 6))
sns.barplot(x=author_counts.index, y=author_counts.values,
hue=author_counts.index, palette="viridis", legend=False)
plt.title('Number of Quotes by Author')
plt.xlabel('Author')
plt.ylabel('Number of Quotes')
plt.xticks(rotation=45)
```

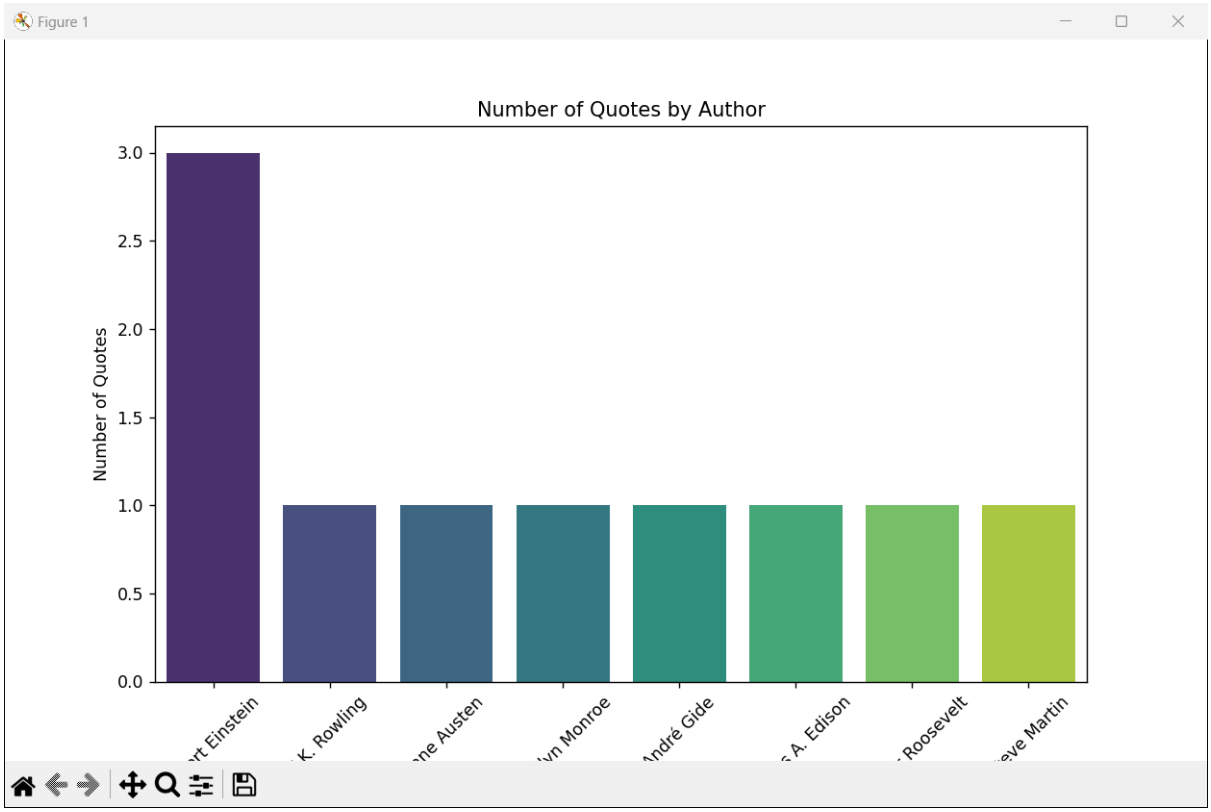
```
plt.show()

# Save the scraped data as a CSV file
data_df.to_csv(r'C:\Users\khana\OneDrive\Desktop\Sana\IT\TY IT\BI\NEw
Practical\scraped_quotes.csv', index=False)

# Print a message to confirm that the data has been saved
print("Scraped data has been saved to 'scraped_quotes.csv'.")
```

OUTPUT:

nc



3. Data Cleaning and Preprocessing (CO/PO: Understand, Analyze; Bloom's: Analyze)
- Task: Clean and preprocess a dataset, handling missing values and outliers using Pandas.
- Normalize or scale numerical features using Scikit-learn.
- Skill: Data cleaning, data transformation.

CODE 1:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler

# Load your dataset from the CSV file
# Replace with the actual file path
file_path = r'C:\Users\khana\OneDrive\Desktop\Sana\IT\TY IT\BI\NEw
Practical\weather_game.csv'

# Read the CSV file with a different encoding (Latin-1)
data = pd.read_csv(file_path, encoding='ISO-8859-1')

#Data Exploration
# Display the first few rows of the dataset
print("First 5 rows of the dataset:")
print(data.head())

# Define a list of numerical columns
numerical_columns = ['Temperature', 'Humidity']

# Summary statistics of the dataset
print("\nSummary statistics of the dataset:")
print(data[numerical_columns].describe())

# Handling Missing Values
# Check for missing values in the numerical columns
missing_values = data[numerical_columns].isnull().sum()
print("\nMissing values in the numerical columns:")
print(missing_values)

data['Temperature'].fillna(data['Temperature'].mean(), inplace=True)

data['Outlook'] = data['Outlook'].astype('category').cat.codes
data['Wind'] = data['Wind'].map({'weak': 0, 'strong': 1})
data['Play'] = data['Play'].map({'no': 0, 'yes': 1})
```

```python
cleaned_file_path = r'C:\Users\khana\OneDrive\Desktop\Sana\IT\TY IT\BI\NEw
Practical\cleaned_weather_data.csv'
data.to_csv(cleaned_file_path, index=False)

#Data Exploration
# Display the first few rows of the dataset
print("\n\nAfter Cleaning\n\n")
print("First 5 rows of the dataset:")
print(data.head())

# Define a list of numerical columns
numerical_columns = ['Outlook','Temperature', 'Humidity','Wind','Play']

# Summary statistics of the dataset
print("\nSummary statistics of the dataset after cleaning:")
print(data[numerical_columns].describe())

# Handling Missing Values
# Check for missing values in the numerical columns
missing_values = data[numerical_columns].isnull().sum()
print("\nMissing values in the numerical columnsafter cleaning:")
print(missing_values)
```

OUTPUT:

```
PS C:\Users\khana> & C:/Users/khana/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/khana/OneDrive/Desktop/Sana/IT/TY IT/BI/NEw Practical/p3_2.py"
First 5 rows of the dataset:
  Outlook  Temperature  Humidity    Wind Play
0   rainy         68.0        72    weak  yes
1   sunny         85.0        98  strong  yes
2   rainy         61.0        44  strong   no
3   rainy         79.0        81  strong   no
4   sunny         87.0        78    weak  yes

Summary statistics of the dataset:
       Temperature   Humidity
count    49.000000  50.000000
mean     78.653061  69.100000
std      13.331590  17.003301
min      60.000000  40.000000
25%      68.000000  53.250000
50%      77.000000  72.000000
75%      85.000000  80.750000
max     130.000000  99.000000

Missing values in the numerical columns:
Temperature    1
Humidity       0
dtype: int64
```

```
After Cleaning


First 5 rows of the dataset:
   Outlook  Temperature  Humidity  Wind  Play
0        1         68.0        72     0     1
1        2         85.0        98     1     1
2        1         61.0        44     1     0
3        1         79.0        81     1     0
4        2         87.0        78     0     1

Summary statistics of the dataset after cleaning:
         Outlook  Temperature   Humidity       Wind       Play
count  50.000000    50.000000  50.000000  50.000000  50.000000
mean    1.000000    78.653061  69.100000   0.480000   0.500000
std     0.782461    13.194852  17.003301   0.504672   0.505076
min     0.000000    60.000000  40.000000   0.000000   0.000000
25%     0.000000    68.000000  53.250000   0.000000   0.000000
50%     1.000000    77.000000  72.000000   0.000000   0.500000
75%     2.000000    85.000000  80.750000   1.000000   1.000000
max     2.000000   130.000000  99.000000   1.000000   1.000000

Missing values in the numerical columnsafter cleaning:
Outlook        0
Temperature    0
Humidity       0
Wind           0
Play           0
dtype: int64
PS C:\Users\khana>
```

CODE 2:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Load your dataset from the CSV file
file_path = r'C:\Users\khana\OneDrive\Desktop\Sana\IT\TY IT\BI\NEw
Practical\weather_game.csv'

# Read the CSV file with a different encoding (Latin-1)
data = pd.read_csv(file_path, encoding='ISO-8859-1')

# Handling Missing Values
data['Temperature'].fillna(data['Temperature'].mean(), inplace=True)

data['Outlook'] = data['Outlook'].astype('category').cat.codes
data['Wind'] = data['Wind'].map({'weak': 0, 'strong': 1})
data['Play'] = data['Play'].map({'no': 0, 'yes': 1})
```

```python
# Save the cleaned data to a new CSV file
cleaned_file_path = r'C:\Users\khana\OneDrive\Desktop\Sana\IT\TY IT\BI\NEw
Practical\cleaned_weather_data.csv'
data.to_csv(cleaned_file_path, index=False)

# Define numerical columns for normalization
numerical_columns = ['Outlook','Temperature', 'Humidity','Wind','Play']

# Min-Max Normalization
min_max_scaler = MinMaxScaler()
data[numerical_columns] =
min_max_scaler.fit_transform(data[numerical_columns])

# Z-score Normalization
z_score_scaler = StandardScaler()
data[numerical_columns] =
z_score_scaler.fit_transform(data[numerical_columns])

# Data Exploration after cleaning and normalization
print("\n\nAfter Cleaning and Normalization\n\n")
print("First 5 rows of the dataset:")
print(data.head())

# Summary statistics of the dataset
print("\nSummary statistics of the dataset after cleaning and normalization:")
print(data.describe())

# Missing values check after cleaning and normalization
missing_values = data.isnull().sum()
print("\nMissing values after cleaning and normalization:")
print(missing_values)
```

OUTPUT:

```
After Cleaning and Normalization


First 5 rows of the dataset:
    Outlook  Temperature  Humidity      Wind  Play
0  0.000000    -0.815562  0.172287 -0.960769   1.0
1  1.290994     0.485900  1.716926  1.040833   1.0
2  0.000000    -1.351458 -1.491171  1.040833  -1.0
3  0.000000     0.026560  0.706970  1.040833  -1.0
4  1.290994     0.639013  0.528742 -0.960769   1.0

Summary statistics of the dataset after cleaning and normalization:
            Outlook   Temperature      Humidity          Wind       Play
count  5.000000e+01  5.000000e+01  5.000000e+01  5.000000e+01  50.000000
mean   8.881784e-18 -1.154632e-16 -2.975398e-16  3.108624e-17   0.000000
std    1.010153e+00  1.010153e+00  1.010153e+00  1.010153e+00   1.010153
min   -1.290994e+00 -1.428014e+00 -1.728808e+00 -9.607689e-01  -1.000000
25%   -1.290994e+00 -8.155618e-01 -9.416359e-01 -9.607689e-01  -1.000000
50%    0.000000e+00 -1.265527e-01  1.722867e-01 -9.607689e-01   0.000000
75%    1.290994e+00  4.858998e-01  6.921172e-01  1.040833e+00   1.000000
max    1.290994e+00  3.930945e+00  1.776335e+00  1.040833e+00   1.000000

Missing values after cleaning and normalization:
Outlook        0
Temperature    0
Humidity       0
Wind           0
Play           0
dtype: int64
PS C:\Users\khana>
```

4. Data Visualization (CO/PO: Create; Bloom's: Apply)
- Task: Create a bar chart or scatter plot using Matplotlib or Seaborn to visualize sales trends over time.
- Skill: Data visualization, chart creation.

CODE 1:

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd

# Load Iris dataset
iris = load_iris()
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
iris_df['species'] = iris.target_names[iris.target]
```

```python
print(iris_df.head())

# Visualizations
plt.figure(figsize=(15, 10))

# Visualization 1: Scatterplot
plt.subplot(2, 4, 1)
sns.scatterplot(x='sepal length (cm)', y='sepal width (cm)', hue='species',
data=iris_df)
plt.title('sepal length (cm) vs sepal width (cm)')

# Visualization 2: Histogram
plt.subplot(2, 4, 2)
sns.histplot(data=iris_df, x='petal length (cm)', hue='species', kde=True)
plt.title('petal length (cm) Distribution')

# Visualization 3: KDE Plot
plt.subplot(2, 4, 3)
sns.kdeplot(data=iris_df, x='petal width (cm)', hue='species')
plt.title('petal width (cm) KDE')

# Visualization 4: Boxplot
plt.subplot(2, 4, 4)
sns.boxplot(x='species', y='sepal width (cm)', data=iris_df)
plt.title('sepal width (cm) Distribution')

# Visualization 5: Violin Plot
plt.subplot(2, 4, 5)
sns.violinplot(x='species', y='sepal length (cm)', data=iris_df)
plt.title('sepal length (cm) Distribution')

# Visualization 6: Jointplot
plt.subplot(2, 4, 6)
sns.jointplot(x='petal width (cm)', y='petal length (cm)', data=iris_df,
kind='scatter')
plt.title('petal width (cm) vs petal length (cm)')

# Visualization 7: PairGrid
plt.subplot(2, 4, 7)
g = sns.PairGrid(iris_df, hue='species', palette='husl')
g.map_upper(sns.scatterplot)
g.map_lower(sns.kdeplot)
g.map_diag(sns.histplot)
g.add_legend()
plt.title('PairGrid')

plt.tight_layout()
plt.show()
```

OUTPUT:

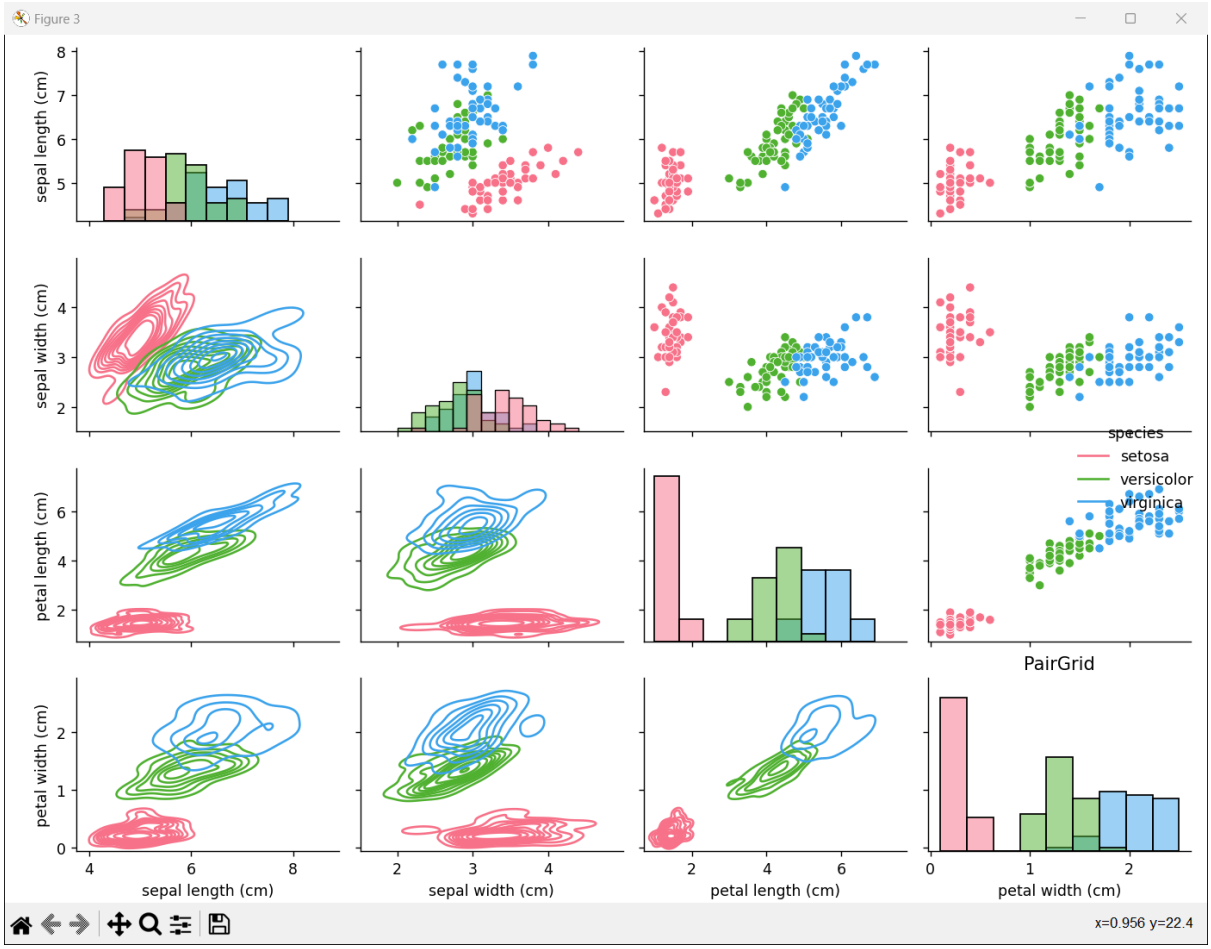


Figure 3
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)
species
setosa
versicolor
virginica
PairGrid
x=0.956 y=22.4

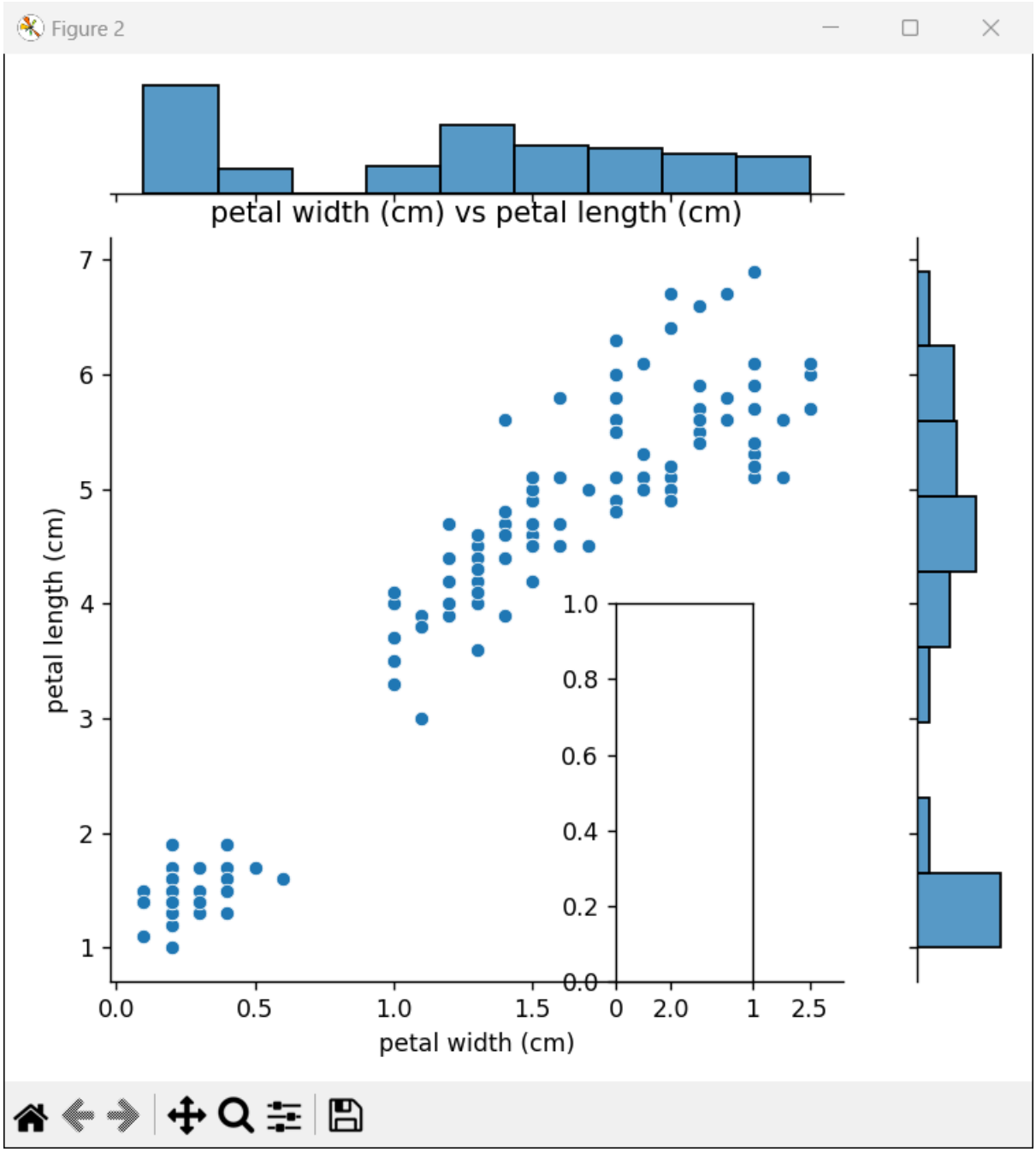
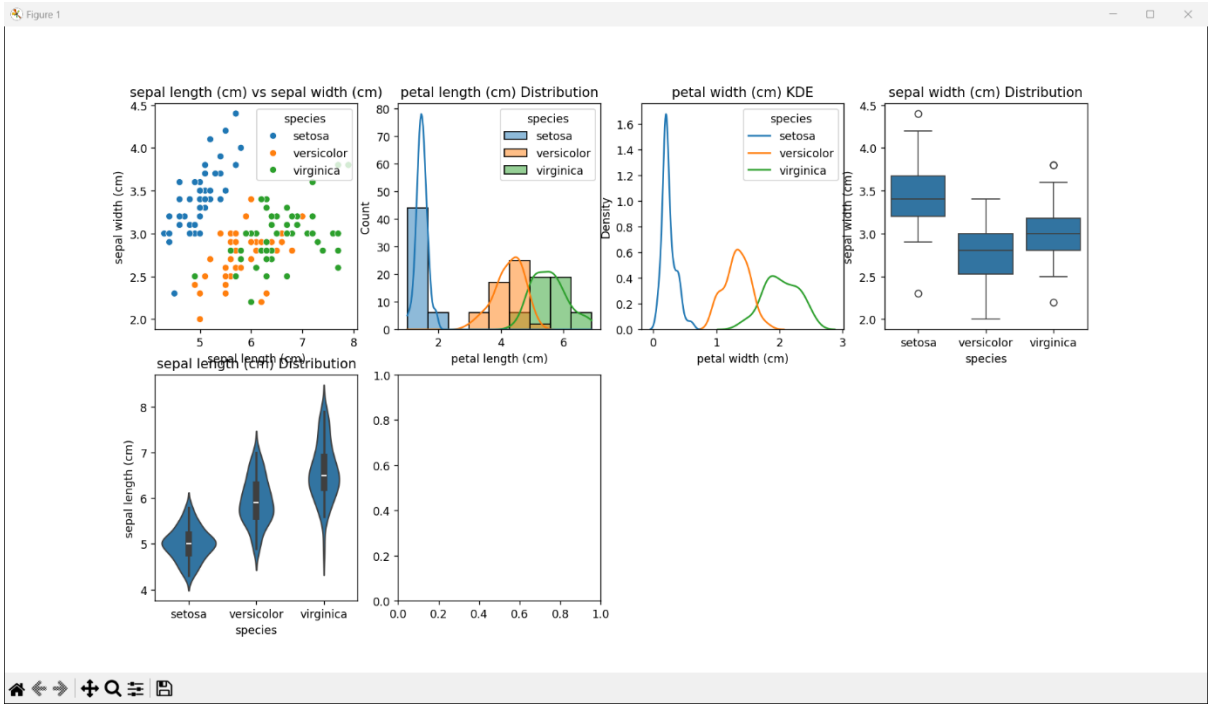


Figure 2
petal width (cm) vs petal length (cm)
petal length (cm)
petal width (cm)
0.0
0.5
1.0
1.5
2.0
2.5
1
2
3
4
5
6
7
0
0.2
0.4
0.6
0.8
1.0



CODE 2:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load your dataset from the URL
url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/00352/Online%20Retail.xlsx"
sales_data = pd.read_excel(url)
print(sales_data.head())
# Adjusting figure size for subplots
plt.figure(figsize=(18, 12))

# Bar Chart of Country-wise Sales
plt.subplot(3, 3, 1)
top_countries = sales_data['Country'].value_counts().head(10)
top_countries.plot(kind='bar')
plt.title('Top 10 Countries by Sales')

# Histogram of Quantity Purchased
plt.subplot(3, 3, 2)
plt.hist(sales_data['Quantity'], bins=30)
plt.title('Distribution of Quantity Purchased')

# Line Plot of Sales Over Time
```

```
plt.subplot(3, 3, 3)
sales_data['InvoiceDate'] = pd.to_datetime(sales_data['InvoiceDate'])
monthly_sales =
sales_data.groupby(sales_data['InvoiceDate'].dt.to_period('M')).size()
monthly_sales.plot(kind='line', marker='o')
plt.title('Monthly Sales Trend')

# Sales Distribution by Product Description
plt.subplot(3, 3, 4)
top_products = sales_data['Description'].value_counts().head(10)
top_products.plot(kind='bar')
plt.title('Top 10 Products by Sales')

# Sales Count by Customer ID
plt.subplot(3, 3, 5)
sales_data['CustomerID'].value_counts().plot(kind='hist', bins=30)
plt.title('Sales Count by Customer ID')

# Sales Variation by Day of the Week
plt.subplot(3, 3, 6)
sales_data['DayOfWeek'] = sales_data['InvoiceDate'].dt.day_name()
sales_by_day = sales_data['DayOfWeek'].value_counts()
sales_by_day.plot(kind='bar')
plt.title('Sales Variation by Day of the Week')

# Monthly Sales Trend
plt.subplot(3, 3, 7)
monthly_sales = sales_data.resample('M', on='InvoiceDate').InvoiceNo.count()
monthly_sales.plot(kind='line', marker='o')
plt.title('Monthly Sales Trend')

# Top Countries by Sales
plt.subplot(3, 3, 8)
top_countries =
sales_data.groupby('Country').size().sort_values(ascending=False).head(10)
top_countries.plot(kind='bar')
plt.title('Top 10 Countries by Sales')

# Sales by Hour of the Day
plt.subplot(3, 3, 9)
sales_data['Hour'] = sales_data['InvoiceDate'].dt.hour
sales_by_hour = sales_data.groupby('Hour').size()
sales_by_hour.plot(kind='bar')
plt.title('Sales by Hour of the Day')

# Adjust layout
plt.tight_layout()
plt.show()
```